Внимание

Начиная с текущей лабораторной работы необходимо **сбрасывать отчёт на почту** koliaulasevich@gmail.com

В теме письма указывается:
«ОС номер группы/подгруппы Фамилия И. О. № лабораторной работы»

Пример
«ОС 3/1 Уласевич Н. И. №1» или «ОС 3/1 Уласевич Н. И. №1,2»

На момент **сдачи 3ей лабораторной** необходимо **сбросить отчеты** по **1ой** и **2ой** лабораторным работам.

Максимально допустимая задержка — одна работа: отчёт по предыдущей лабораторной должен быть отправлен не позднее дня сдачи следующей. Например, если 4-я работа сдана 26.09.2026, то отчёт по 3-й должен быть отправлен в этот же день.

Лабораторная работа №3
Управление процессами

Цель работы: Получение практических навыков в работе с процессами в ОС Windows и Linux.

!!!Задания являются накопительными, т.е. для выбранного уровня сложности нужно выполнить также все предыдущие.

Темы для предварительного изучения:

- Утилита Sysinternals Process Explorer (ссылка);
- Работа с процессами в Windows (лекция и документация) и POSIX (лекция и документация);
- Работа с IPC (лекция, Linux, Windows)

Постановка задачи для Windows:
Уровень 4-5:

Приложение Lab-03x:

Программа выполняет простейший цикл с заданным количеством итераций, при чём каждая итерация выполняется с задержкой 500 мс. Программа должна выводить PID на каждой итерации и перед стартом выводить количество итераций. Количество итераций передается в программу через аргументы командной строки. Если не задан аргумент командной строки, то количество итераций приложение пытается получить через переменную окружения ITER_NUM. Если не заданы ни переменная окружения, ни аргумент командной строки приложение завершает свою работу с ошибкой (Использовать **ExitProcess**).

Приложение Lab-03a:

Программа создаёт три дочерних процесса с помощью **CreateProcess**. Все три процесса должны создаваться для программы **Lab-02x**, при чём для передачи имени исполняемого файла (в данном случае используется полное имя файла) и параметра количества итераций должны быть выполнены следующие условия:

- При создании первого процесса используется только первый параметр функции **CreateProcess**. Количество итераций также требуется указать в первом параметре, так задумано (прим. «**Lab-03x.exe 50**»).
- При создании второго процесса используется только второй параметр функции **CreateProcess**. По сути взять строку из предыдущего пункта и передать во второй параметр.
- При создании третьего процесса для передачи имени программы используется – первый параметр, для количества итераций – второй. (при этом во втором параметре вам может понадобиться пробел в начале строки)

Родительский процесс должен дождаться завершения всех дочерних процессов (выполняются одновременно), перед тем как завершиться самому.

***Вопрос**: Все ли процессы созданы успешно? Если нет, почему?*

Также предусмотреть обработку ошибок и очистку ресурсов. Использовать Process Explorer, cmd и TaskManager для просмотра информации о созданных процессах. В Process Explorer проверить наличие дескрипторов дочерних процессов у родительского процесса.

Установить глобальную переменную окружения ITER_NUM для вашего пользователя.

Приложение Lab-03b:

Создать программу идентичную программе Lab-03a, но:

- В первом вызове **CreateProcess** оставить только имя файла (т.е. убрать количество итераций, которое указывалось в Lab-03a).
- В третьем вызове передать NULL вместо второго параметра, и перед этим установить локальную переменную окружения ITER_NUM. Значение должно отличаться от глобальной переменной.

***Вопрос:** Все ли процессы созданы успешно? Сколько итераций выполнил каждый процесс и почему?*

Приложение Lab-03c:

Создать программу, которая выводит список запущенных в системе процессов. Для каждого процесса выводится его имя, идентификатор и родительский идентификатор. (Tool Help Library)

Уровень 7-8:

Приложение Lab-03d-client:

Программа выполняет поиск чисел, взаимно простых с заданным числом M, в указанном диапазоне. Диапазон и число M задаются через аргументы командной строки. Программа перебирает все числа n в диапазоне [L, R], проверяет что n взаимно простое с M (наибольший общий делитель 1) и сохраняет найденные числа в промежуточном буфере. После завершения работы цикла отправляется по анонимному каналу родительскому процессу.
Считать, что $M > 0$ и $L \geq 1$.

Структура вызова данного приложения:

Lab-03d-client <Число M> <Нижний порог диапазона> <Верхний порог диапазона>

Предусмотреть обработку ошибок и корректную очистку всех ресурсов.

Приложение Lab-03d-server:

Программа запускает заданное количество дочерних процессов для поиска чисел, взаимно простых с M. Также программа создаёт анонимный канал для работы с дочерними процессами (передача дескриптора через

STARTUPINFO). При получении информации от дочернего процесса программа должна выводить её на консоль.

Структура вызова данного приложения:

Lab-03d-server <Количество создаваемых процессов> <Число M> <Нижний порог диапазона> <Верхний порог диапазона>

Сервер выводит:

– Число M.
– Диапазон [L, R].
– Количество процессов K.
– Общее количество найденных чисел.
– Сами найденные числа (в одну строку через пробел).
– Время работы

Указанный диапазон [L, R] делится на равные части в количестве, равном количеству создаваемых процессов. Каждый дочерний процесс обрабатывает свой поддиапазон.

Программа не может быть завершена пока не завершены все дочерние процессы.

Предусмотреть обработку ошибок и очистку ресурсов.

Использовать Process Explorer и найти дескриптор канала в момент работы приложения.

Примечание! Никакой синхронизации предусматривать на данном этапе не требуется!

Вопрос: *Какой тип дескриптора указан в Process Explorer для канала? Почему?*

Уровень 9-10:

Разработать два приложения: родительский и дочерний процессы, реализующие механизм передачи данных через структуру STARTUPINFO. Родительский процесс при создании дочернего передаёт через STARTUPINFO фразу «Hello, my dear child!», которую должен получить создаваемый процесс и выводить на консоль!

Примечание! Запрещено использовать переменные окружения, перенаправлять потоки ввода/вывода, передавать данные через аргументы командной строки дочернему процессу, использовать какие-либо стандартные механизмы IPC!

Постановка задачи для Linux:

Уровень 4-5:

Приложение Lab-03x:

Программа является полным аналогом программы из заданий для Windows.

Приложение Lab-03a:

Программа создаёт два дочерних процесса и загружает в них исполняемый файл **Lab-03x**. При создании процессов должны быть выполнены следующие условия:

- В ходе запуска первого дочернего процесса параметр с количеством итераций должен быть передан как аргумент командной строки
- В ходе запуска второго дочернего процесса параметр с количеством итераций должен быть передан через переменную окружения ITER_NUM. Переменная создается непосредственно в родительском процессе перед созданием второго дочернего процесса

Родительский процесс должен дождаться завершения всех дочерних процессов (выполняются одновременно).

Изучить информацию о созданных процессах через файловую систему (каталог /proc) и утилиту ps. Предусмотреть обработку ошибок и очистку ресурсов.

***Вопрос**: Все ли процессы созданы успешно? Если нет, почему?*

Уровень 7-8:

Написать аналог приложений **Lab-03d-*** для ОС Linux. Предусмотреть обработку ошибок и очистку ресурсов.

Уровень 9-10:

Написать приложение, которое ведёт себя также, как представлено на следующем скриншоте:

```
$> ./signals &
[1] 25831
$> Program running with PID 25831

$> kill -USR1 25831
Received USR1 signal
$> kill -USR1 25831
Received USR1 signal
$> kill -USR2 25831
$> kill -USR2 25831
Received USR2 signal
$> Ctrl+C
^C
$> kill -USR1 25831
Received USR1 signal
$> kill -TERM 25831
$> ENTER
[1]+  Terminated              ./signals
```

Обращаю внимание, что Ctrl+C не способно остановить работу такого приложения!

Вопросы для самоконтроля:

1. Что такое процесс? Чем процесс отличается от программы?
2. Какие ресурсы доступны процессу?
3. Что такое дочерний и родительский процесс? Как они связаны?
4. Что такое системный вызов? Зачем он нужен?
5. Какие системные вызовы предназначены для создания процессов в Windows? Linux?
6. Какие функции из WinAPI и POSIX связаны с ранее названными системными вызовами?
7. Сколько новых процессов появится в результате выполнения программой следующей последовательности вызовов **fork**() (учитывая, что все они выполнятся успешно)?

fork();
fork();
fork();

8. Для чего нужен **exec**?
9. Что такое межпроцессное взаимодействие (IPC)? **(7-8)**
10. Какие группы IPC-механизмов вам известны? Какие механизмы входят в эти группы? **(7-8)**
11. Какой механизм взаимодействия выбрать, когда требуется максимальная скорость передачи данных? **(7-8)**
12. Какой механизм взаимодействия выбрать, когда требуется повышенная устойчивость к ошибкам? **(7-8)**
13. Какие сигналы существуют в Linux? Приведите примеры и укажите их назначение. **(9-10)**
14. Как обеспечить обработку сигнала в процессе? **(9-10)**
15. Назовите структуры ядер ОС Windows и Linux описывающие процесс. **(9-10)**
16. Назовите способы просмотреть их структуру. **(9-10)**